

React Day 7

Hooks in React

Hooks are special functions in React that let you **use features like state, lifecycle, and context** inside **function components**.

Before hooks, these features were only available in **class components**. But now with hooks, we can do everything using **function components**.

useEffect Hook Kya Hota Hai?

useEffect ek React Hook hai jo humein side effects chalane deta hai jab component render hota hai.

Side effects ka matlab:

- Data fetch karna (API call)
- DOM update karna
- Event listener lagana ya hataana
- Timer chalana (setInterval / setTimeout)
- Local storage update karna

Basic use of useEffect → prevent extra rendering of component on state (V.imp)

Syntax

useEffect (pass function as a parameter , dependancy)

```
useEffect(()=>{  
  | //write your code  
},[props or state])
```

```

1  useEffect(() => {
2    //call every time
3  })
4
5
6  useEffect(() => {
7    //call only once
8  },[])
9
10
11  useEffect(() => {
12    //call on changing single state
13  },[state1])
14
15
16  useEffect(() => {
17    //call on changing both state
18  },[state1, state2])
19
20
21  useEffect(() => {
22    //call on changes props
23  },[prop1, prop2])

```

- In projects what we have done
 - state side effect handling
 - props side effect handling
-

Components Life Cycle in React

what is human lifeCycle ?

- birth → middle → death

same as this react components also have 3 phases

- mounting → updating → unmounting
- these are three major phases in react component

```
useEffect(() => {  
  // This code will run on mount  
},[])  
  
useEffect(() => {  
  // This code will run on update  
},[state1 or props])  
  
useEffect(() => {  
    
  return () => {  
    // This code will run on unmount  
  };  
}, []);  
  
useEffect(() => {  
  // This code will run on every life cycle event  
})
```

Styling React using CSS (5).

- inline style
- external style
- css modules
- styled component
- external CSS library/Framework

Point	 External CSS	 CSS Modules
 Scope	Global – har component ko affect kar sakta hai	Local/Scoped – sirf us component tak limited
 Class Name Conflict	Ho sakta hai agar same class name bar-bar use ho jaye	Nahi hota – har class ka naam internally unique
 Import Syntax	import './App.css'	import styles from './App.module.css'
 Usage in JSX	className="heading"	className={styles.heading}
 Accidental Overwrite	Ho sakta hai (last loaded CSS rule apply hoti hai)	Nahi hota – sab kuch scoped hota hai
 Code Maintainability	Thoda tough in large projects	Easy & scalable
 Reusability of class names	Risky – same name repeat karna dangerous	Safe – har component me same naam use kar sakte ho
 Team Project Friendly?	✗ Nahi – conflict ka chance hota hai	✓ Haan – no tension of naming clashes
 Setup Effort	Simple – normal CSS likhna	Slightly extra – .module.css ka syntax use karna padta hai
✓ Recommended for?	Small / personal projects	Medium to large / team projects